Deployment of streamlit app on AWS

STEPS

1) Create a ML Model

2) Create an Streamlit App On Local

3) Containerize – Docker

4) Push the Docker image to Docker hub

5) Create an AWS EC2 instance

6) Pull the Docker image from Docker hub to AWS EC2

7) Start the Docker and access the streamlit app

Directory Structure

```
iris_deployment/
│
├── app.py              # Streamlit app
├── model.pkl           # saved ML model
├── train_model.py      # training script
├── requirements.txt
├── Dockerfile
└── README.md
```

Create a ML Model

1) Goto the project folder and right click and open command prompt:

Create a ML Model

Install necessary packages:

1) **python –m pip install scikit-learn pandas numpy**

2) **python –m pip install streamlit**

3) Create **train_model.py** with code shown on right.

4) **python train_model.py**

```python
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import pickle

# Load data
X, y = load_iris(return_X_y=True)

# Train model
model = RandomForestClassifier()
model.fit(X, y)

# Save model
pickle.dump(model, open("model.pkl", "wb"))

print("Model saved!")
```

Create streamlit app

1) Write following in **app.py**

2) Run the app.py:

python –m streamlit run app.py

3) Goto local URL:

http://localhost:8501

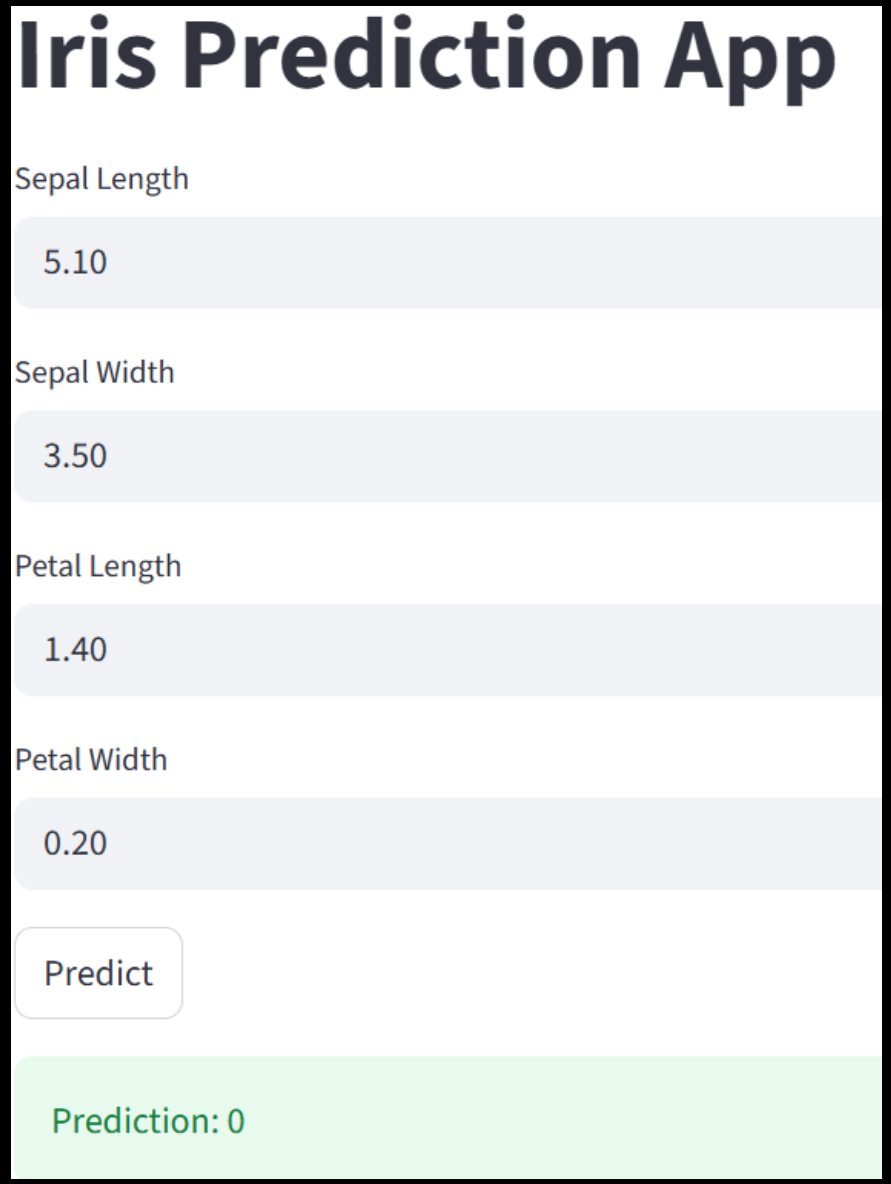
```python
import streamlit as st
import pickle
import numpy as np

model = pickle.load(open("model.pkl", "rb"))

st.title("Iris Prediction App")

sl = st.number_input("Sepal Length")
sw = st.number_input("Sepal Width")
pl = st.number_input("Petal Length")
pw = st.number_input("Petal Width")

if st.button("Predict"):
    prediction = model.predict([[sl, sw, pl, pw]])
    st.success(f"Prediction: {prediction[0]}")
```

1) Enter following data to test if it is working:

Sepal length = 5.1
Sepal Width = 3.5
Petal Length = 1.4
Petal Width = 0.2

Output should be **0**

2) Go back to command shell
and kill the the process: **ctrl + c**



Model has been successfully deployed locally.

Dockerize streamlit app

1) Create requirements.txt file

See the file **requirements.txt**

```
streamlit
scikit-learn
numpy
pandas
```

Dockerize streamlit app

Create Dockerfile:

```
FROM python:3.12 (last pushed 4 hours ago)


# specify Current Working Directory
WORKDIR /code

# copy requirements.txt from local to container
COPY ./requirements.txt /code/requirements.txt

# install all requirements in the container
RUN pip install --no-cache-dir --default-timeout=100 -r /code/requirements.txt

# copy dir app from local to container
COPY . /code

# Expose port 8501 so other app can communicate with container
EXPOSE 8501

# Run following command to start app
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0"]
```

3) Docker commands

step a: Start docker engine

Start the docker engine by simply running the **Docker Desktop**: Start -> Docker Desktop

step b: Build docker image

Run following command in **same directory** which contains Dockerfile:

> docker build -t image_ml_model .

This would pull the image from docker hub. This could take **4 - 10 minutes**.

step c: Run container

> docker run --name container_ml_model -p 8501:8501 image_ml_model

NOTE: If you make a mistake, then delete the **container_ml_model**

> docker rm -f <container_id_name>

and redo above steps

Dockerize streamlit app

4) Goto URL http://127.0.0.1:8501

You should see the streamlit app

5) Enter following data →

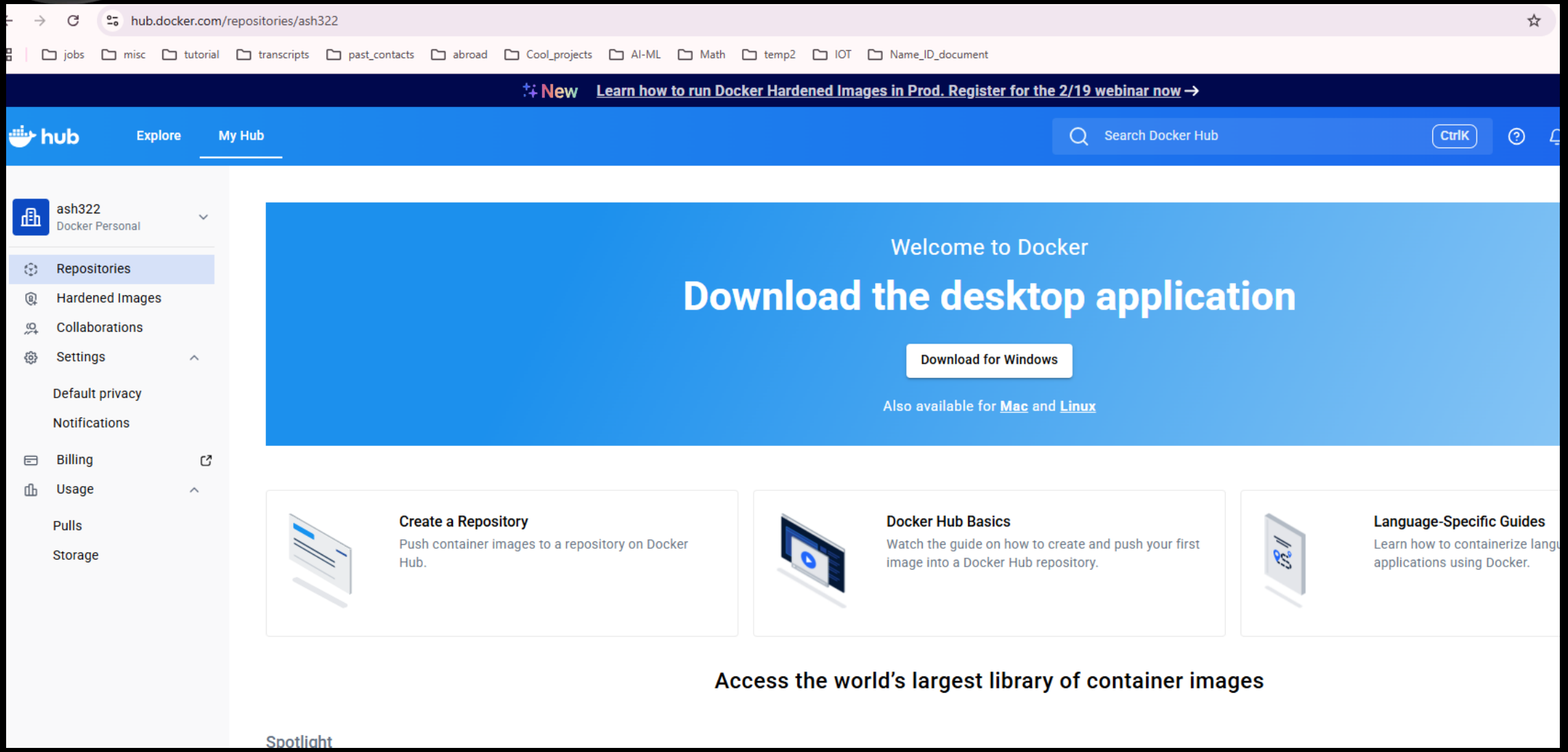
Here the docker is running locally

Now you have successfully dockerized your streamlit app

Push the Docker Image to Docker Hub

Push the Image to Docker Hub

Goto docker hub and create an account. Also note down user name



Push the Image to Docker Hub

Create a repository: **image_ml_model**

hub.docker.com/repository/create?namespace=ash322

jobs | misc | tutorial | transcripts | past_contacts | abroad | Cool_projects | AI-ML | Math | temp2 | IOT | Name_ID_document

New Learn how to run Docker Hardened Images in Prod. Register for the 2/19 webinar now →

hub | Explore | My Hub | Search Docker Hub | CtrlK

ash322
Docker Personal

- Repositories
- Hardened Images
- Collaborations
- Settings
  - Default privacy
  - Notifications
- Billing
- Usage
  - Pulls
  - Storage

Repositories / Create

Using 0 of 1 private reposi

Create repository

Repository Name *
iris-model

Short description
This is an image that contains iis model

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. Get more

- Public — Appears in Docker Hub search results
- Private — Only visible to you

Cancel | Create

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

Push the Image to Docker Hub

1) Now let's look at the docker images on local machine:

> **docker image ls**

2) Now we tag the image:
> **docker tag image_ml_model ash322/image_ml_model**

Here **ash322** is my username

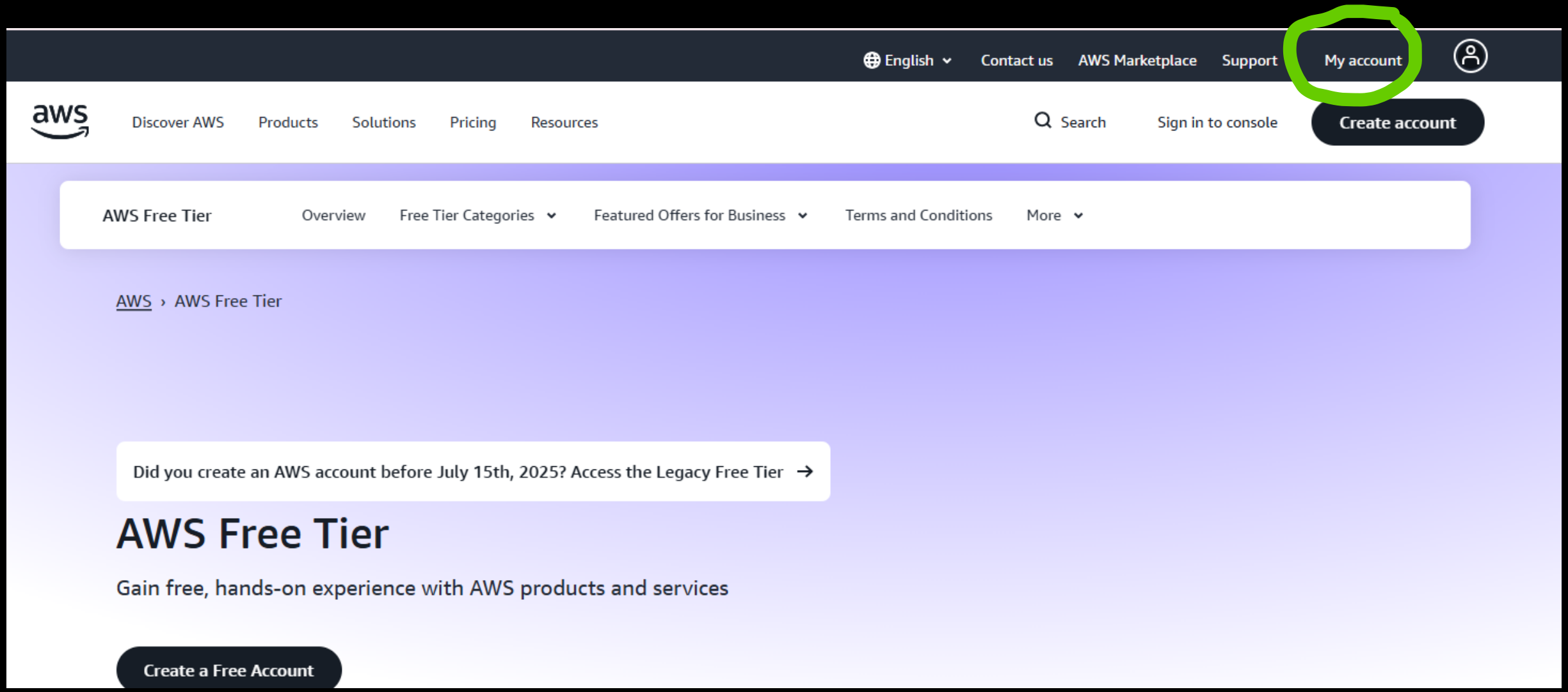
3) Now login:
> **docker login**

(Follow instructions)

3) Now push the image to docker hub (This could take few minutes):
> **docker push ash322/image_ml_model**

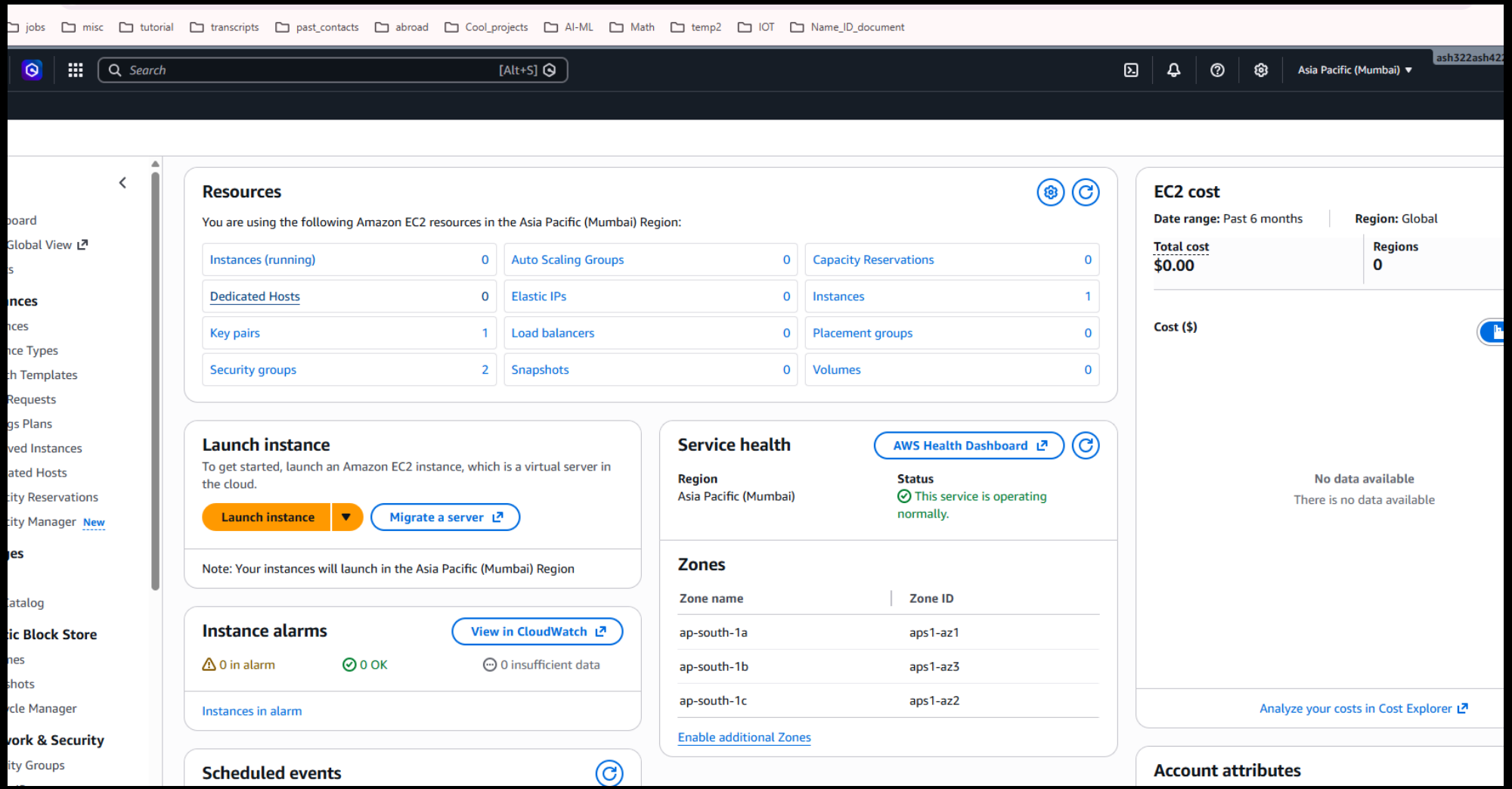
NOW CREATE AN AWS EC2 INSTANCE AND PULL THE DOCKER IMAGE

Login to AWS account.
If you do not have one, then sign up. It is free



Here we **launch the EC2 instance**.

- Name = iris-model-instance
- OS image=Ubuntu
- Do not need to create key-pair. Proceed without key-pair



Now we connect to our EC2 instance: **Connect using a Public IP**

Search [Alt+S] Asia Pacific (Mumbai) ▾ ash322ash422

EC2 > Instances > i-02c06c5d64146ea03 > Connect to instance

Connect Info

Connect to an instance using the browser-based client.

Instance ID	VPC ID	Security groups	IAM role
i-02c06c5d64146ea03 (image-temp)	vpc-05d5014b44e8bfb33	sg-0daa70c0ec529a816 (launch-wizard-2)	–

EC2 Instance Connect | SSM Session Manager | SSH client | EC2 serial console

Instance ID
i-02c06c5d64146ea03 (image-temp)

Connection type

- (●) Connect using a Public IP
  Connect using a public IPv4 or IPv6 address
- ( ) Connect using a Private IP
  Connect using a private IP address and a VPC endpoint

(●) **Public IPv4 address**
65.2.172.161

( ) **IPv6 address**
–

Username
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ubuntu.

ubuntu

Note: In most cases, the default username, ubuntu, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Cancel | Connect

You should see following EC2 console

ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-02c06c5d64146ea03&osUser=ubuntu&region=ap-south-1&sshPort=22

jobs | misc | tutorial | transcripts | past_contacts | abroad | Cool_projects | AI-ML | Math | temp2 | IOT | Name_ID_document | All Bookmar

aws | Search [Alt+S] | Asia Pacific (Mumbai) | ash322ash422 (3814-9199-2660) admin

S3

```
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-32-61:~$
```

i-02c06c5d64146ea03 (image-temp)

PublicIPs: 65.2.172.161 PrivateIPs: 172.31.32.61

Run following commands on **EC2 console:**

Install Docker on EC2

> sudo apt update

> sudo apt install docker.io -y

> sudo systemctl start docker

Check if docker is running: **ps –aux | grep dock**

Login to docker: **docker login**

Go to url specified and paste the code

(Now we give permission)

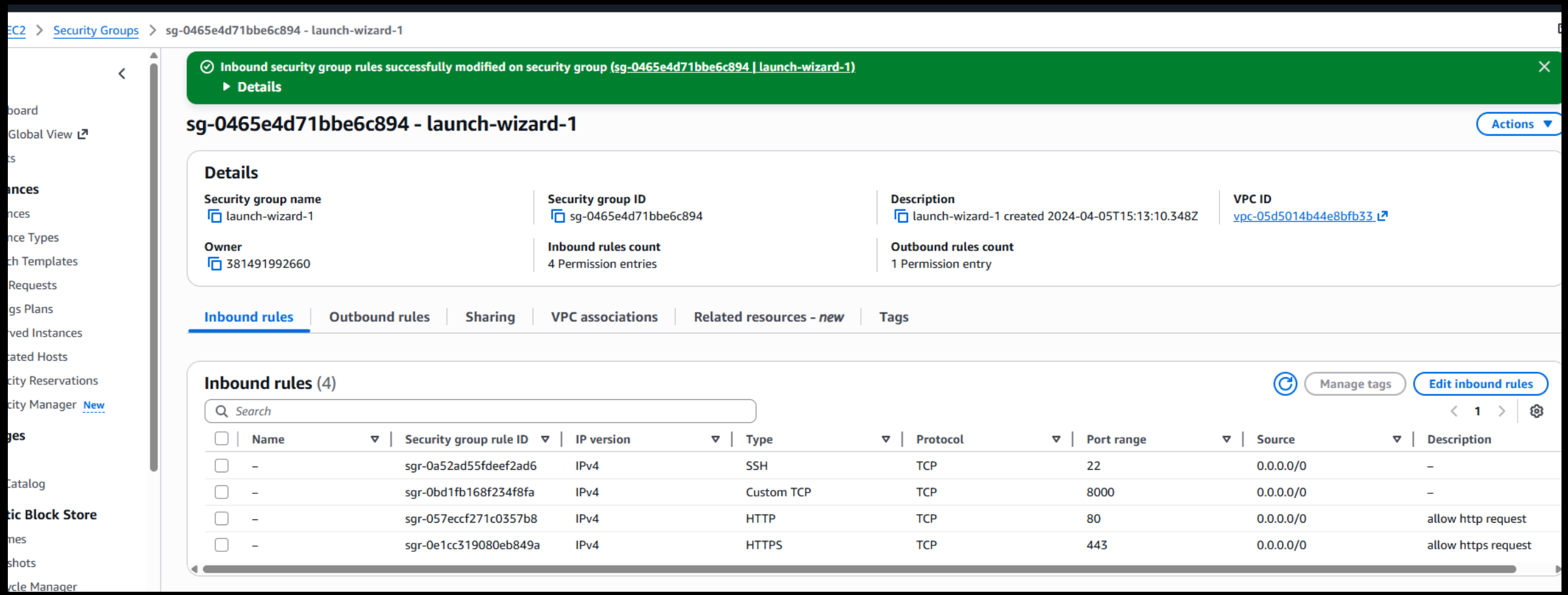
> sudo usermod -aG docker $USER

> newgrp docker

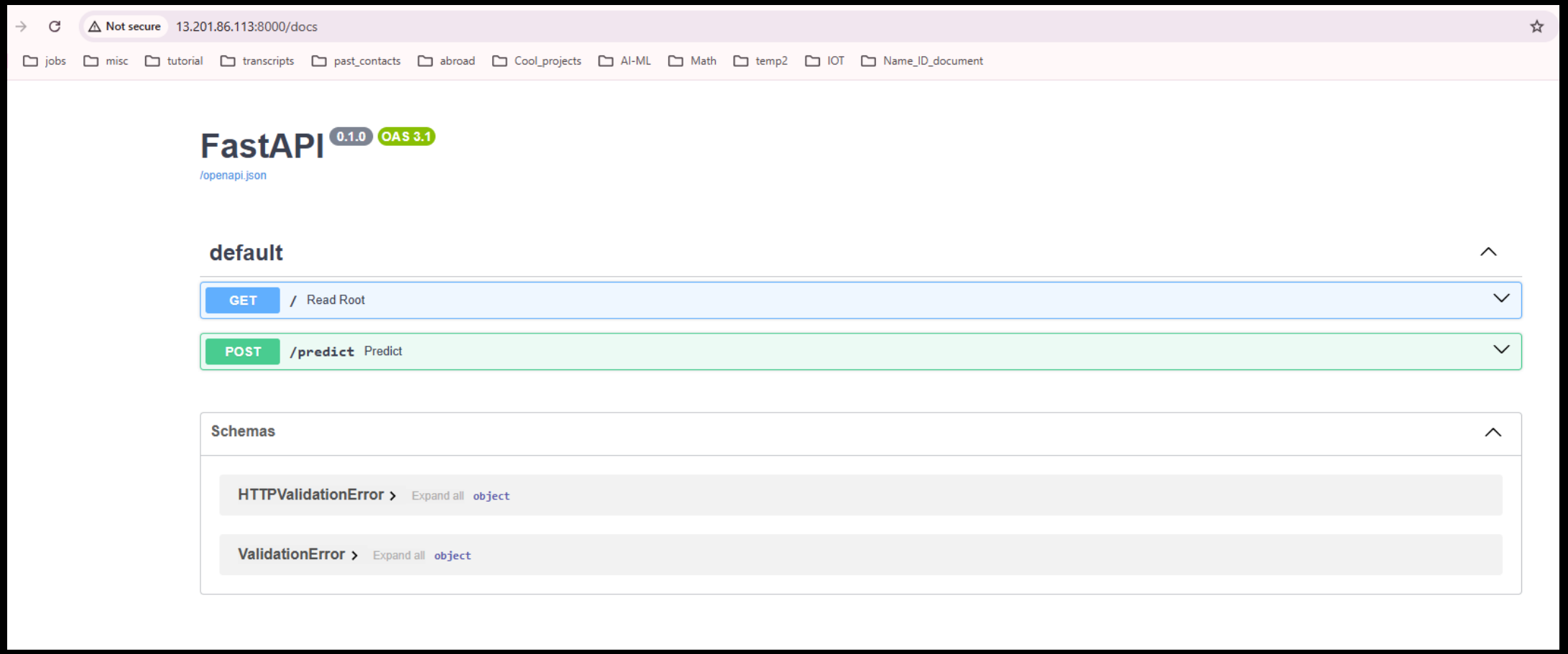
Pull Image on Ubuntu OS: **docker pull ash322/image_ml_model**

Run Container: **docker run -d -p 8501:8501 ash322/image_ml_model**

Open Security Group:
Make sure you have following
Custom TCP → Port 8501 → Anywhere (0.0.0.0/0)



Access Model from the internet

http://EC2-PUBLIC-IP:8501

Your ML model is LIVE on the internet.